Domain-Specific RAG Chatbot

Student Name: Nishiyath P A
Batch: Artificial-Intelligence-August-B-2026-batch

Project Objective

To develop a Retrieval-Augmented Generation (RAG) chatbot that answers questions from uploaded documents. The system retrieves relevant document passages, generates answers using a language model, and displays source document names and page numbers.

Methodology

1. **PDF Extraction:** Extracted text from PDF documents using **pypdf**, preserving page numbers.
2. **Text Cleaning and Chunking:** Cleaned extracted text and divided it into smaller chunks.
3. **Embeddings:** Converted text chunks into numerical representations using the Sentence Transformers model **all-MiniLM-L6-v2**.
4. **Vector Search:** Used **FAISS** to store and retrieve relevant document chunks.
5. **Answer Generation:** Connected the retrieved context and user question to **Groq** using a context-based prompt.
6. **User Interface:** Developed a **Streamlit application** for PDF uploads, document processing, question answering, and source display.

Technologies Used

Python, pypdf, LangChain Text Splitters, Sentence Transformers, FAISS, Groq API, Streamlit, and python-dotenv.

Project Outcome

The application processes PDFs uploaded. It creates text chunks and embeddings, retrieves relevant information, generates answers, and displays source references. A testing sheet was prepared with 15 questions to evaluate the chatbot.

Limitations

The chatbot currently supports text-based PDFs, not scanned documents. Answer quality depends on document extraction and retrieval, and generated answers may require verification against the original notes.

Conclusion

This project provided practical experience in building a RAG pipeline by connecting document processing, embeddings, vector search, language-model integration, and a Streamlit interface. It demonstrates how uploaded documents can be used as a searchable knowledge source for question answering.